

LLMsVerifier

LLMsVerifier

Verifizieren. Überwachen. Optimieren.

Zusammenfassung

LLMsVerifier ist eine Enterprise-Plattform zur Verifizierung, Überwachung und Optimierung von Large Language Models verschiedener Anbieter. Sie basiert auf einem obligatorischen „Siehst du meinen Code?“-Verifikationstest, sodass nur Modelle, die nachweislich funktionieren, als nutzbar markiert oder exportiert werden.

Kurzbeschreibung

Eine Go-Plattform, die LLMs über mehrere Anbieter hinweg verifiziert, benchmarkt, überwacht und optimiert. Jedes Modell muss einen obligatorischen Code-Sichtbarkeitstest bestehen, bevor es eingesetzt wird. Anschließend durchläuft es Latenz-, Streaming-, Funktionsaufruf-, Bildverarbeitungs- und Embedding-Prüfungen. Nur verifizierte Konfigurationen werden für AI- und CLI-Tools exportiert.

Ausführliche Beschreibung

LLMsVerifier ist eine umfassende Plattform zur Verifizierung, Überwachung und Optimierung der Leistung von LLM-Modellen verschiedener Anbieter. Ihr zentrales Prinzip lautet *obligatorische Verifizierung* – und sie setzt es kompromisslos durch: Bevor ein Modell als nutzbar markiert oder in eine exportierte Konfiguration übernommen wird, muss es einen „Siehst du meinen Code?“-Test bestehen, der echte HTTP-Aufrufe an den Anbieter sendet und die Antwort auf echte Verständnisfähigkeit analysiert – nicht auf eine

plausibel wirkende Echo-Antwort. Ein Modell, das Ihre Eingabe nicht nachweislich sieht und versteht, erhält niemals das „nutzbar“-Flag.

Nach dieser Hürde führt die Verifizierungs-Engine eine vollständige Reihe von Fähigkeitstests durch – Existenz, Reaktionsfähigkeit, Latenz, Streaming, Funktionsaufrufe, Bildverarbeitung und embeddings. Die Reporter-Engine wandelt die Ergebnisse in Markdown- und JSON-Berichte um, auf deren Grundlage Sie Maßnahmen ergreifen können.

Das System ist modular und ereignisgesteuert aufgebaut und bietet CLI-, TUI-, Web- sowie REST- und API-Schnittstellen, die auf einem Kern aus Verifizierungs-Engine, Reporter-Engine und Konfigurationsmanager basieren. Doch LLMsVerifier geht über die reine Verifizierung hinaus: Erweiterte Schichten implementieren ein Supervisor/Worker-Muster für LLM-gestützte Aufgabenzerlegung, ein gleitendes Fenster mit LLM-Zusammenfassung für die Kontextverwaltung, sodass sehr lange Sitzungen nicht abrupt abbrechen, cloudbasierte Checkpoints sowie ein Failover-System mit Circuit Breakern und latenzbasiertem Routing.

Die umgebende Infrastruktur ist produktionsreif gestaltet: ein Pub/Sub-Ereignisbus, Cron-Scheduling, Erkennung von Preisen und Limits, eine vector-Datenbank für RAG sowie ein Exportsystem. Eine charakteristische Markierungskonvention fügt jedem generierten Anbieter/Modell das Suffix (llmsvd) hinzu, sodass verifizierte Ausgaben auf einen Blick erkennbar sind und niemals mit ungeprüften verwechselt werden können. Nur verifizierte Modelle werden in exportierte Konfigurationen für AI- und CLI-Tools wie OpenCode, Crush und Claude Code übernommen.

Die Plattform liefert das operative Tooling, das Teams in der Produktion tatsächlich benötigen: Docker-/Kubernetes-/Helm-Bereitstellung, Prometheus-/Grafana-Überwachung, LDAP/SSO sowie SQLCipher-verschlüsselte Speicherung.

Warum wir es entwickelt haben

Weil reine Konfigurationsprüfungen unzuverlässig sind – ein API-Schlüssel kann ablaufen, ein Modell veraltet sein, und eine Konfigurationsdatei sagt nichts über echte Latenz, echte Fehler oder darüber aus, ob das Modell Ihre Eingabe überhaupt sieht und versteht. LLMsVerifier ersetzt das Prinzip „Es steht in der Konfiguration, also muss es funktionieren“ durch Nachweis: Nur Modelle, die nachweislich korrekt antworten, werden als nutzbar markiert und exportiert.

Warum es ein Game-Changer ist

Es macht LLM-Flotten *vertrauenswürdig* – ein Wort, das in einem Bereich, in dem Konfigurationen oft durch Auslassungen lügen, selten verdient wird. Statt darauf zu hoffen, dass ein konfiguriertes Modell funktioniert, erhalten Teams eine durchgesetzte, testbare Garantie, dass jedes eingesetzte Modell echte Verifizierung durchlaufen hat. Überwachung, Failover-Mechanismen und der Export ausschließlich verifizierter Modelle schließen den Kreis von der Validierung bis zur Produktion. Innerhalb des Helix-Ökosystems wird es zur einzigen Quelle der Wahrheit für LLM-Modelle, Anbieter und Verifizierungsmetadaten: Andere Dienste (darunter HelixTranslate) richten sich danach aus, sodass eine gesamte Plattform eine einzige ehrliche Antwort auf die Frage erhält: *„Welche Modelle funktionieren gerade wirklich?“* – statt dass jedes Team seine eigene optimistische Vermutung pflegt.

Was innovativ ist

- **Verpflichtende „Siehst du meinen Code?“-Verifizierung** – eine echte, HTTP-gestützte Verständnisprüfung, die ein Modell bestehen muss, bevor es überhaupt nutzbar ist; das prägende Alleinstellungsmerkmal des Produkts und der Grund, warum nichts Ungeprüftes durchrutscht.
- **Export ausschließlich verifizierter Konfigurationen** – generierte Konfigurationen für AI-CLI-Tools enthalten *nur* Modelle, die die Verifizierung bestanden haben, sodass die ausgelieferte Konfiguration nicht heimlich ein fehlerhaftes Modell wieder einführt.
- **(llmsvd)-Branding-Suffix-System** – jeder generierte Anbieter und jedes Modell trägt ein nachverfolgbares Suffix, das die verifizierte Herkunft überall sichtbar macht, wohin die Ausgabe gelangt.
- **Fähigkeitserkennung** über zahlreiche CLI-Agenten und -Anbieter hinweg – es erfasst Streaming-Typen (SSE, WebSocket, JSONL, EventStream), Komprimierung und Caching-Verhalten, statt sie einfach vorauszusetzen.
- **Robustes Failover** – Schutzschalter, latenzbasiertes Routing, das bei Überschreiten einer Schwelle für die Zeit bis zum ersten Token umleitet, Gesundheitsprüfungen und gewichtete Traffic-Verteilung halten eine Flotte reaktionsfähig, selbst wenn einzelne Anbieter instabil werden.
- **Langfristige Autonomie** – ein Supervisor/Worker-Zerlegungsmuster in Kombination mit Checkpointing und Speicherintegration ermöglicht ausgedehnte Sitzungen, die sonst an Kontextgrenzen scheitern würden.
- **RAG-/vector-DB-Integration** zur Anreicherung des Kontexts mit fundierten Daten.

Größte technische Herausforderungen & wie wir sie gelöst haben

- **Beweisen, dass ein Modell tatsächlich funktioniert - und nicht nur konfiguriert ist.** Genau das ist der Kern und zugleich die größte Hürde. Gelöst durch den verpflichtenden Code-Sichtbarkeitstest, der echte API-Aufrufe durchführt und die Antworten auf tatsächliche Verständnissfähigkeit analysiert, gestützt auf eine breite Palette von Fähigkeitstests - und anschließend durch die Weigerung, etwas zu exportieren, das nicht bestanden hat. So entscheidet nicht die Konfiguration, sondern der Nachweis über die Produktionsfreigabe.
- **Zuverlässigkeit trotz zahlreicher unzuverlässiger Drittanbieter.** Gelöst mit einem Failover-Orchestrator, der Anbieterinstabilität als Normalfall behandelt: Schutzschalter markieren einen Anbieter nach N Fehlern in M Sekunden als beeinträchtigt, latenzbasiertes Routing lenkt von langsamen Endpunkten weg, regelmäßige Gesundheitschecks prüfen auf Erholung, und gewichtetes Routing balanciert kostengünstige mit Premium-Modellen.
- **Aufrechterhaltung sehr langer, autonomer Sitzungen.** Gelöst durch das Supervisor/Worker-Zerlegungsmuster, das große Aufgaben in handhabbare Teile aufspaltet, periodisches Checkpointing in Cloud-Speicher, damit Fortschritte Unterbrechungen überstehen, und ein mehrschichtiges Kontextmanagement (gleitendes Fenster + LLM-Zusammenfassung + RAG), sodass das Modell den roten Faden behält, ohne in Tokens zu ertrinken.
- **Anbieter-Wildwuchs.** Gelöst, indem zahlreiche anbieterspezifische Go-Adapter hinter einer gemeinsamen Schnittstelle verborgen werden, wobei die eigentlichen Endpunkte zentral verwaltet werden - sodass das Hinzufügen eines Anbieters eine isolierte Änderung bleibt und nicht durch den gesamten Codebase hallt.

Technologie-Stack

- **Go** — als Kernplattformsprache aufgrund ihrer Nebenläufigkeit ausgewählt; sie treibt eine multithreaded Verifier-Engine an, die viele Modelle parallel prüfen kann, sowie die umliegenden Dienste.
- **Gin** — als REST-API-Server gewählt, der JWT-Authentifizierung, Ratenbegrenzung sowie WebSocket-/SSE-Endpunkte bereitstellt.
- **SQLite + SQLCipher** — für eingebettete Speicherung mit verschlüsselter Datenbank gewählt, da Verifizierungsdaten (Schlüssel, Ergebnisse) sensibel sind und standardmäßig im Ruhezustand verschlüsselt werden sollten.

- **Redis** — als Caching-Schicht ausgewählt, um schnelle Abfragen von Verifizierungsdaten und Metadaten zu gewährleisten.
- **RabbitMQ + Kafka** — zur Unterstützung der ereignisgesteuerten Architektur gewählt: Nachrichtenübermittlung und Streaming, die Produzenten und Konsumenten innerhalb der Plattform entkoppeln.
- **gRPC + Protocol Buffers** — für stark typisierte Kommunikation zwischen Diensten und den Transport von Ereignissen zwischen Komponenten ausgewählt.
- **QUIC / HTTP-3 (quic-go)** — für moderne Transportunterstützung gewählt (die Repository-Dokumentation weist darauf hin, dass die Verfügbarkeit von HTTP/3-Anbietern begrenzt ist — eine angebotene Funktion, keine universelle Aussage).
- **JWT + LDAP/NTLM** — für Unternehmensauthentifizierung gewählt, damit sich die Plattform nahtlos in bestehende Unternehmensidentitäten (SSO/SAML/OIDC) einfügt (laut Dokumentation unterstützt).
- **Viper (Konfiguration), Logrus (Logging), Brotli/compress (Komprimierung)** — die operativen Grundlagen: flexible Konfiguration, strukturierte Protokollierung und Payload-Komprimierung.
- **Angular** — für die Web-Einzelanwendung ausgewählt, das visuelle Frontend für Verifizierung und Überwachung.
- **Python + JavaScript-SDKs** — gewählt, um Entwicklungsteams erstklassigen Zugriff zu bieten, dokumentiert über OpenAPI/Swagger.
- **Docker, Kubernetes, Helm** — für die Produktionsbereitstellung mit Gesundheitsüberwachung und automatischer Skalierung gewählt, damit sich ein Verifizierungs-Cluster wie jeder moderne Dienst skalieren lässt.
- **Prometheus + Grafana** — für Metriken und Dashboards ausgewählt, um die Gesundheit der Plattform selbst genauso beobachtbar zu machen wie die Modelle, die sie überwacht.
- **Testify (Go) + node -test/jsdom (Web)** — für mehrschichtiges Testen des Go-Kerns und des Web-Frontends gewählt.

Status & Transparenzhinweise

- **Status: Beta.** Der Go-Quellcode implementiert echte HTTP-Verifizierung (ein veraltetes Dokument, das Verifizierung als reine Konfiguration beschreibt, ist lediglich ein Wunschbild — der Code ist maßgeblich).
- **Lizenz: Noch nicht festgelegt.** Die README gibt MIT an, während ein Dockerfile-Label Apache-2.0 nennt — vor der Veröffentlichung klären.

- Anbieteranzahl: Die README spricht von „12 Adaptern“, doch das *providers*-Verzeichnis listet etwa 26 auf — als „12+ / weitere in Arbeit“ behandeln. Zahlreiche Dateien mit dem Status „FINAL/COMPLETE“ existieren; maßgeblich sind Code, Dokumentation und `go.mod`.
- Das Repository befindet sich in der *vasic-digital*-Organisation, fungiert jedoch funktional als Vertrauensschicht des Helix-LLM-Infrastrukturclusters.

Prioritätsstufe: Helix-primär (LLM-Infrastrukturcluster; einzige Quelle der Wahrheit für LLM-/Anbieter-/Verifizierungsmetadaten). Rangiert nach HelixTrack.